

V MAX Protocol

Transport	TCP, ใช้การเชื่อมต่อแบบ Persistent Connection เพียงหนึ่งการเชื่อมต่อสำหรับแต่ละโหนด
Encoding	รูปแบบข้อความ โดยใช้ Header ที่ได้รับแรงบันดาลใจจาก HTTP และ JSON เป็นรูปแบบของ Body
Version token	VMP/1.0

1. ภาพรวมและวัตถุประสงค์

VMP (V MAX Protocol) เป็นโพรโทคอลระดับแอปพลิเคชัน (Application-Layer Protocol) ที่ออกแบบขึ้นสำหรับรองรับการรับส่งข้อมูล Telemetry และ Control ในเครือข่ายเซนเซอร์ IoT ด้านการเกษตรที่จำลองขึ้น โดยโหนดเซนเซอร์ (Sensor Node) ที่กระจายอยู่ตามแปลงเกษตรจะเชื่อมต่อกับเซิร์ฟเวอร์ส่วนกลาง (Central Server) ผ่าน TCP และทำงานในรูปแบบ Hybrid กล่าวคือ โหนดจะส่งค่าที่ตรวจวัดได้ไปยังเซิร์ฟเวอร์ตามรอบเวลาของตนเอง ขณะที่เซิร์ฟเวอร์สามารถส่งคำสั่งกลับไปยังโหนดใดก็ได้ผ่านการเชื่อมต่อเดียวกันได้ตลอดเวลา โดยไม่มีช่องทาง Polling แยกต่างหาก และไม่มีการใช้ Message Broker เป็นตัวกลาง

ระบบจำลองโหนดเซนเซอร์ทั้งหมด 3 ประเภท โดยแต่ละประเภทใช้สำหรับตรวจวัดค่าทางกายภาพที่แตกต่างกัน ดังนี้

1. **TempHumidNode** วัดอุณหภูมิอากาศ (°C) และความชื้นสัมพัทธ์ (%) โดยข้อมูลทั้งสองค่าจะถูกส่งเป็นชุดเดียวกัน โดยจำลองการทำงานจากเซนเซอร์ระดับ DHT22

2. **SoilNode** วัดค่า pH ของดินและความชื้นในดิน (%)

3. **LightNode** วัดความเข้มแสง มีหน่วยเป็น ลักซ์ (lux)

วงจรการทำงานของโหนดมีลำดับดังนี้

1. เชื่อมต่อ
2. REGISTER
3. ส่งข้อมูล PUSH เป็นระยะ (และตรวจสอบสถานะด้วย STATUS เป็นครั้งคราว)
4. ตอบสนองต่อ COMMAND ที่เซิร์ฟเวอร์ส่งมา
5. UNREGISTER เมื่อปิดการทำงานอย่างถูกต้อง

ทุกข้อความที่แต่ละฝ่ายส่งหรือได้รับจะถูกแสดงบน Console พร้อม รหัสสถานะ (Status Code) และ ข้อความอธิบายสถานะ (Status Phrase) แบบเต็ม ตามข้อกำหนดด้านการบันทึก Log ของงาน (Logging Requirement)

2. เหตุผลที่เลือกใช้ TCP แทน UDP

โดยทั่วไปการรับส่งข้อมูล Telemetry ในระบบ IoT มักนิยมใช้ UDP เนื่องจากมี Overhead ต่ำกว่า และเคยพิจารณาการใช้สถาปัตยกรรมแบบ TCP+UDP Hybrid โดยให้ UDP รับผิดชอบการส่งข้อมูลจากโหนดไปยังเซิร์ฟเวอร์ และใช้ TCP สำหรับการส่งคำสั่งจากเซิร์ฟเวอร์กลับไปยังโหนด อย่างไรก็ตาม VMP เลือกใช้ TCP สำหรับการสื่อสารทั้งระบบ ด้วยเหตุผล 3 ประการ ดังนี้

1. ความถูกต้องของการบันทึกสถานะและข้อมูล

ข้อกำหนดของงานระบุว่าต้องแสดง Status Code และ Status Phrase ของทุกข้อความที่ส่งและได้รับ TCP รับประกันการส่งข้อมูลที่ครบถ้วนและเรียงลำดับถูกต้อง (In-order, Complete Delivery) ทำให้ข้อมูลที่ถูกบันทึกใน Log สอดคล้องกับสิ่งที่เกิดขึ้นจริงบนเครือข่าย

ในทางกลับกัน UDP อาจเกิดการสูญหายของ Packet ได้ ซึ่งจะทำให้ Log อาจไม่สอดคล้องกับข้อมูลที่ถูกส่งจริง หากไม่มีระบบจัดการเพิ่มเติมเพื่อบันทึกและตรวจสอบ Packet ที่สูญหาย

2. คำสั่งจากเซิร์ฟเวอร์ต้องได้รับอย่างถูกต้อง

คำสั่งจากเซิร์ฟเวอร์ไปยังโหนด เช่น "เพิ่มความถี่ในการส่งข้อมูล (Increase Push Frequency)" จำเป็นต้องได้รับอย่างแน่นอน หากใช้ UDP แล้ว Datagram สูญหาย โหนดอาจไม่ได้รับ คำสั่งและยังคงทำงาน ด้วยการตั้งค่าเดิมโดยที่เซิร์ฟเวอร์ไม่ทราบถึงปัญหา

3. ความน่าเชื่อถือของข้อมูลจากเซนเซอร์

ข้อมูลจากเซนเซอร์ถูกนำไปใช้ประกอบการตัดสินใจด้านการให้น้ำและการใส่ปุ๋ย ในบริบทของระบบที่จำลองขึ้น ดังนั้นการสูญหายของข้อมูลโดยที่ระบบไม่ตรวจพบจึงเป็นสิ่งที่ไม่พึงประสงค์

เหตุผลที่ไม่เลือก TCP+UDP Hybrid

สถาปัตยกรรมแบบ TCP+UDP Hybrid ถูกปฏิเสธ เนื่องจากจะเพิ่มความซับซ้อนในการจัดการ Socket โดยแต่ละโหนดต้องรองรับการสื่อสารผ่าน Transport Protocol ถึง 2 ประเภท แม้ UDP จะมี Overhead ต่ำกว่า แต่ข้อได้เปรียบดังกล่าวมีความสำคัญน้อยกว่าสำหรับระบบที่มีขนาดตามที่กำหนด เมื่อเทียบกับความเสี่ยงที่ข้อมูลหรือ Status Log จะไม่สอดคล้องกับเหตุการณ์ที่เกิดขึ้นจริง ดังนั้น TCP จึงเป็นตัวเลือกที่เหมาะสมกว่าในบริบทของ VMP ทั้งในด้านความน่าเชื่อถือ ความถูกต้องของข้อมูล และความง่ายในการออกแบบระบบ

3. รูปแบบข้อความ (Message Format)

ข้อความทุกข้อความของ VMP ใช้รูปแบบ Text-based โดยประกอบด้วย Start Line ที่ได้รับแรงบันดาลใจจาก HTTP ตามด้วย Header ในรูปแบบ Key: Value จากนั้นเป็นบรรทัดว่าง และ JSON Body โดยกำหนดความยาวของ Body ที่แน่นอนเป็นจำนวนไบต์ผ่าน Header Content-Length

รูปแบบดังกล่าวเป็นการนำแนวคิด HTTP/1.1 Message Framing มาใช้โดยตรง ซึ่งเป็นเนื้อหาที่ได้ศึกษาในรายวิชา แทนที่จะออกแบบโครงสร้างข้อมูลแบบไบนารีขึ้นมาใหม่

PUSH VMP/1.0

Node-ID: TEMP-01

Seq: 4

Content-Length: 78

```
{"temperature":29.7,"humidity":63.8,"timestamp":"2026-08-30T14:30:00+07:00"}
```

รูปที่ 1 โครงสร้างของ Request Message

การออกแบบรูปแบบข้อความนี้มีเหตุผลสำคัญ 2 ประการ ดังนี้

1. ใช้ JSON Body แทน Binary Struct โหนดเซนเซอร์ทั้ง 3 ประเภทมีชุดข้อมูลที่แตกต่างกัน เช่น อุณหภูมิและความชื้น, ค่า pH และความชื้นในดิน หรือค่าความเข้มแสง (lux) การใช้ JSON ช่วยให้ แต่ละประเภท สามารถส่งข้อมูลในโครงสร้างที่เหมาะสมกับตนเองได้ โดยไม่จำเป็นต้องเปลี่ยนแปลงรูปแบบ Header นอกจากนี้ JSON ยังสามารถอ่านและตรวจสอบได้โดยตรงจาก Terminal Log ซึ่งสอดคล้องกับข้อกำหนดของงาน และไม่จำเป็นต้องเขียนโค้ดสำหรับการจัดการตำแหน่ง Byte Offset ด้วยตนเอง
2. ใช้ Content-Length สำหรับกำหนดขอบเขตของข้อความ แทนการใช้ JSON แบบแบ่งด้วยบรรทัดใหม่ (Newline-delimited JSON) TCP มีลักษณะเป็น Byte Stream และไม่มีขอบเขตของแต่ละข้อความ (Message Boundary) ในตัว หากใช้ \n เป็นตัวแบ่งข้อความ อาจเกิดปัญหาได้หากข้อมูล JSON มีอักขระขึ้นบรรทัดใหม่อยู่ภายใน การระบุความยาวของ Body ที่แน่นอนล่วงหน้าด้วย Content-Length เช่นเดียวกับ HTTP จึงช่วยกำจัดการคลุมเครือนี้ โดยฝั่งรับจะอ่าน Header จนกว่าจะพบ Blank Line จากนั้นอ่านข้อมูล Body ตามจำนวนไบต์ที่ระบุใน Content-Length และถือว่าข้อความนั้นสมบูรณ์

สำหรับ Response Message จะใช้โครงสร้างเดียวกัน แต่เปลี่ยนจาก Method Line เป็น Status Line ในรูปแบบที่คล้ายกับ HTTP ดังตัวอย่าง

VMP/1.0 **200 OK**

Content-Length: 40

```
{"message":"Push received successfully"}
```

รูปที่ 2 โครงสร้างของ Response Message

4. รูปแบบการทำงานพร้อมกัน (Concurrency Model)

เซิร์ฟเวอร์จำเป็นต้องรองรับการเปิดการเชื่อมต่อจากโหนดหลายตัวพร้อมกัน โดยโหนดที่ผ่านการ REGISTER แล้วจะคงการเชื่อมต่อ Socket ไว้ เพื่อให้เซิร์ฟเวอร์สามารถส่ง COMMAND ไปยังโหนดนั้นได้ทุกเมื่อ โดยการเชื่อมต่อที่ทำงานช้าหรืออยู่ในสถานะ Idle จะต้องไม่ส่งผลกระทบต่อการทำงานของการทำงานของการเชื่อมต่ออื่น

เซิร์ฟเวอร์อ้างอิงของ VMP จึงเลือกใช้ Event-Driven, Single-Threaded I/O Model ของ Node.js แทนการสร้าง Thread ใหม่สำหรับแต่ละ Connection

เมื่อมีการเชื่อมต่อเข้ามา แต่ละ Connection จะถูกแทนด้วย Object ประเภท socket ผ่าน Event Handler `server.on('connection', ...)` และเมื่อโหนดผ่านกระบวนการ REGISTER แล้ว ระบบจะจับคู่ Node-ID ของโหนดเข้ากับ Socket ที่ยังเปิดใช้งานอยู่ โดยจัดเก็บไว้ในตารางการเชื่อมต่อฝั่งเซิร์ฟเวอร์ในรูปแบบ `Map<Node-ID, RegisteredNode>`

ด้วยวิธีนี้ เซิร์ฟเวอร์สามารถค้นหา Socket ที่ยังเชื่อมต่ออยู่ของโหนดใด ๆ ได้โดยตรงเมื่อจำเป็นต้องส่ง COMMAND

แนวทางดังกล่าวสอดคล้องกับรูปแบบการทำงานของ Node.js โดยไม่จำเป็นต้องจัดการ Thread Lifecycle ด้วยตนเอง และ Event Loop ของ Node.js สามารถจัดการและ Multiplex การรับส่งข้อมูลผ่าน Socket หลายรายการที่เกิดขึ้นพร้อมกันได้อยู่แล้ว ซึ่งเหมาะสมกับ Workload ของระบบ VMP ที่มีลักษณะเป็น I/O-bound เป็นหลัก

5. เมธอด (Methods)

VMP กำหนดเมธอดทั้งหมด 5 เมธอด ได้แก่ REGISTER, PUSH, STATUS, COMMAND และ UNREGISTER

- REGISTER, PUSH และ STATUS ใช้สำหรับการสื่อสารจาก โหนดไปยังเซิร์ฟเวอร์ (Node > Server)
- COMMAND ใช้สำหรับการสื่อสารจาก เซิร์ฟเวอร์ไปยังโหนด (Server > Node)
- UNREGISTER ใช้สำหรับการสื่อสารจาก โหนดไปยังเซิร์ฟเวอร์ (Node > Server)

5.1 REGISTER ลงทะเบียนโหนด

ส่งเพียงครั้งเดียวทันทีหลังจากสร้างการเชื่อมต่อ และต้องดำเนินการก่อนที่จะสามารถส่ง PUSH ได้ ข้อความ REGISTER ประกอบด้วย Node-ID, Node-Type และ Plot-ID และสามารถระบุ Auth-Token เพิ่มเติมได้ เมื่อ REGISTER สำเร็จ เซิร์ฟเวอร์จะผูก Identity ของโหนด เข้ากับ TCP Connection นั้นในตารางการเชื่อมต่อของเซิร์ฟเวอร์ และจะคงการผูกนี้ไว้ตลอดอายุการใช้งานของ Socket

5.2 PUSH รายงานค่าที่ตรวจวัดได้

เป็นข้อความที่โหนดเป็นผู้เริ่มส่งตามช่วงเวลาที่กำหนด (Node-initiated) โดยประกอบด้วย Header Seq และ JSON Body ที่มีโครงสร้างแตกต่างกันไปตามประเภทของโหนด

เซิร์ฟเวอร์จะยอมรับ PUSH เฉพาะจากโหนดที่ได้ทำการ REGISTER บน Connection นั้นเรียบร้อยแล้วเท่านั้น

5.3 COMMAND ส่งคำสั่งไปยังโหนด

เป็นข้อความที่เซิร์ฟเวอร์เป็นผู้เริ่มส่ง (Server-initiated) โดยส่งผ่าน Connection เดิมที่โหนดได้ REGISTER ไว้แล้ว VMP กำหนดประเภทของคำสั่ง (Subtype) ทั้งหมด 5 ประเภท

Subtype	ผลลัพธ์ (Effect)	Payload
SET_INTERVAL	เปลี่ยนความถี่ในการส่งข้อมูลของโหนด	{"seconds": n}
REPORT_NOW	สั่งให้โหนดส่งค่าที่ตรวจวัดล่าสุดทันที โดยไม่ต้องรอรอบถัดไป	{}
SET_THRESHOLD	กำหนดให้โหนดส่งข้อมูลก่อนถึงรอบปกติ เมื่อค่าที่กำหนดข้าม Threshold ขั้นต่ำ เช่น ความชื้นในดินต่ำเกินไป	{"field": s, "min": n}
CALIBRATE	ปรับค่า Offset ของการอ่านจากเซนเซอร์	{"offset": n}
SHUTDOWN	สั่งให้โหนดทำการ UNREGISTER และยกเลิกการเชื่อมต่อ	{}

เซิร์ฟเวอร์มี Operator REPL ซึ่งช่วยให้ผู้ปฏิบัติงานสามารถส่งคำสั่งเหล่านี้ได้แบบ Interactive โดยสามารถกำหนดเป้าหมายเป็น Node-ID รายตัว หรือ Plot-ID ทั้งแปลง

5.4 STATUS ตรวจสอบสถานะ

เป็น Request ที่มี Body ว่างและประกอบด้วย Node-ID เพียงอย่างเดียวโดย เซิร์ฟเวอร์จะตรวจสอบว่า โหนดดังกล่าวยังมีอยู่ใน Connection Table หรือไม่

- หากพบโหนด > ตอบกลับ 200 พร้อม {"registered":true}
- หากไม่พบโหนด > ตอบกลับ 401

Client อ้างอิงจะส่ง STATUS โดยอัตโนมัติทุก ๆ 3 ครั้งของ PUSH เพื่อใช้ในการตรวจสอบสถานะอย่างง่าย โดยอาศัยจังหวะเดียวกับการส่ง PUSH ที่มีอยู่แล้ว แทนที่จะสร้าง Timer ใหม่สำหรับ STATUS

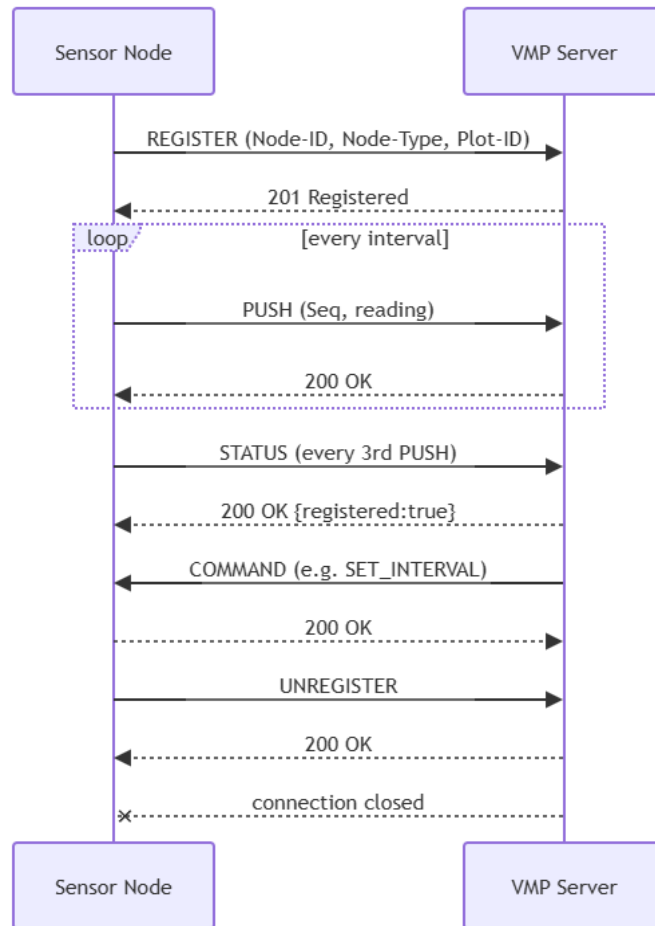
5.5 UNREGISTER ยกเลิกการลงทะเบียน

เป็น Request ที่มี Body ว่างและเริ่มต้นโดยโหนด (Node-initiated) เมื่อเซิร์ฟเวอร์ได้รับ UNREGISTER จะดำเนินการดังนี้

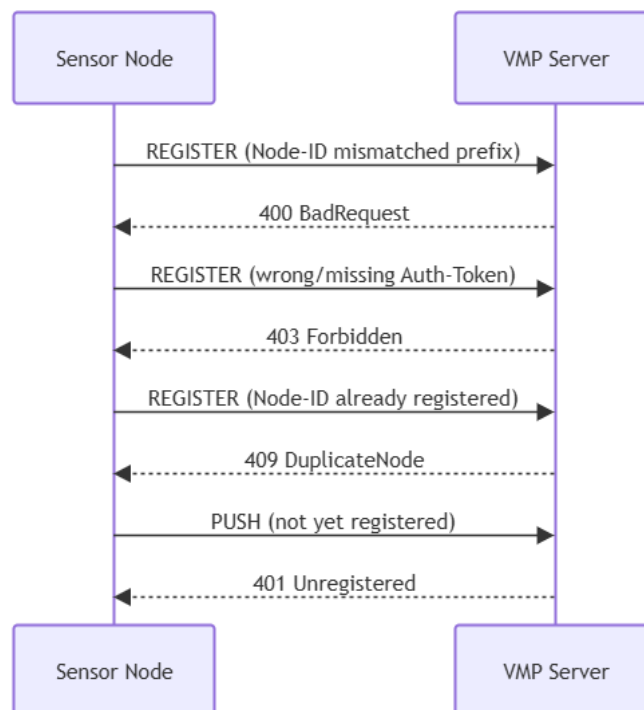
1. ลบโหนดออกจาก Connection Table
2. ตอบกลับด้วย Status 200
3. ปิด Socket Connection

UNREGISTER จะถูกส่งเฉพาะเมื่อมีการยกเลิกการเชื่อมต่อโดยตั้งใจ เช่น การกด Ctrl+C หรือเมื่อได้รับคำสั่ง SHUTDOWN

ในกรณีที่การเชื่อมต่อสิ้นสุดลงโดยไม่ได้ตั้งใจ โหนดจะ ไม่ส่ง UNREGISTER แต่เซิร์ฟเวอร์จะตรวจพบการสิ้นสุดของ Connection ผ่าน Event close หรือ error ของ Socket โดยตรง



รูปที่ 3 ลำดับการทำงานของ Session ตั้งแต่เริ่มต้นจนสิ้นสุด (Happy Path)



รูปที่ 4 ลำดับการทำงานในกรณีเกิดข้อผิดพลาดระดับ 4xx (4xx Error Paths)

6. รหัสสถานะ (Status Code)

VMP กำหนด Status Code และ Status Phrase เพื่อระบุผลลัพธ์ของแต่ละข้อความ ที่มีการรับส่ง ระหว่างโหนด และเซิร์ฟเวอร์ ดังนี้

Code	Phrase	ความหมาย
200	OK	การดำเนินการสำเร็จทั่วไป เช่น PUSH ได้รับการยอมรับ, COMMAND ได้รับการตอบรับ, STATUS ได้รับการยืนยัน หรือ UNREGISTER ดำเนินการเสร็จสมบูรณ์
201	Registered	REGISTER ได้รับการยอมรับ และโหนดถูกผูกเข้ากับ Connection นี้เรียบร้อยแล้ว
400	BadRequest	Request ไม่ถูกต้อง เช่น Header ที่จำเป็นของ REGISTER ขาดหาย, Prefix ของ Node-ID ไม่ตรงกับ Node-Type หรือใช้ Version Token ของ VMP ในรูปแบบที่ไม่รองรับ (§9)
401	Unregistered	มีการส่ง PUSH หรือ STATUS จาก Node-ID ที่ไม่มีอยู่ใน Connection Table ของเซิร์ฟเวอร์
403	Forbidden	REGISTER มี Auth-Token ที่ขาดหายหรือไม่ถูกต้อง ในกรณีที่เซิร์ฟเวอร์เปิดใช้งานการป้องกันด้วย Secret (§8)
404	NodeNotFound	สงวนไว้สำหรับกรณีที่มีการอ้างอิงเป้าหมายไปยัง Node-ID ที่เซิร์ฟเวอร์ไม่รู้จัก
409	DuplicateNode	REGISTER ด้วย Node-ID ที่ถูกลงทะเบียนอยู่แล้วบน Connection อื่น
500	InternalError	สงวนไว้สำหรับข้อผิดพลาดที่ไม่คาดคิดซึ่งเกิดขึ้นภายในฝั่งเซิร์ฟเวอร์

7. ความน่าเชื่อถือ: Seq และการเชื่อมต่อใหม่อัตโนมัติ

TCP รับประกันการส่งข้อมูลที่ ครบถ้วนและเรียงลำดับถูกต้องภายใน Connection เดียวกัน แต่ไม่สามารถรับประกันความต่อเนื่องของข้อมูล เมื่อมีการตัดการเชื่อมต่อและสร้าง Connection ใหม่ หาก Connection ของโหนดหลุดและมีการเชื่อมต่อใหม่ Connection ใหม่จะไม่มีข้อมูลหรือสถานะของ Connection เดิม ทำให้ข้อมูลที่อยู่ระหว่างการส่งในขณะที่ Connection หลุดอาจสูญหาย และ ทั้งฝั่งโหนด และเซิร์ฟเวอร์จะไม่สามารถทราบได้โดยตรงว่าข้อมูลดังกล่าวสูญหาย VMP จึงมีคุณสมบัติ 2 ประการเพื่อจัดการกับปัญหานี้

7.1 Seq Header

Seq เป็นค่าจำนวนเต็มที่กำหนดไว้ในทุกข้อความ PUSH โดยเริ่มต้นที่ 1 และจะรีเซ็ตกลับเป็น 1 ทุกครั้งที่มีการ REGISTER ใหม่ รวมถึงกรณีที่เกิดการเชื่อมต่อใหม่ (Reconnect) เซิร์ฟเวอร์จะเก็บค่า Seq ล่าสุดที่ได้รับของแต่ละโหนดไว้ โดยมีหลักการตรวจสอบดังนี้

- หากค่า Seq กระโดดข้ามไปข้างหน้า > บันทึก Log ว่าอาจเกิดข้อมูลสูญหาย (Likely Gap)
- หากค่า Seq ซ้ำกับค่าที่เคยได้รับ หรือมีค่าน้อยกว่าหรือเท่ากับค่าล่าสุด > บันทึก Log ว่าเป็นข้อมูลซ้ำ (Duplicate)

อย่างไรก็ตาม Seq มีหน้าที่เพียง ตรวจสอบและบันทึกเหตุการณ์ (Detection and Logging) เท่านั้น ไม่ได้ทำหน้าที่เป็นชั้นสำหรับส่งข้อมูลซ้ำ (Retransmission Layer) ดังนั้น แม้จะตรวจพบ Gap หรือ Duplicate เซิร์ฟเวอร์ยังคงประมวลผลข้อความและตอบกลับด้วย 200 ตามปกติ เพื่อให้การออกแบบ ยังอยู่ภายใน ขอบเขตของข้อกำหนดด้าน Logging ของงาน

7.2 การเชื่อมต่อใหม่อัตโนมัติด้วย Exponential Backoff

เมื่อเกิดการตัดการเชื่อมต่อโดยไม่ได้ตั้งใจ เช่น Socket เกิด Event close หรือ error Client จะไม่สิ้นสุดการทำงานทันที แต่จะพยายามเชื่อมต่อกับเซิร์ฟเวอร์ใหม่โดยใช้ Exponential Backoff ตามลำดับ $1s > 2s > 4s > 8s > \dots$

โดยมีการกำหนดค่าสูงสุด (Cap) เพื่อป้องกันไม่ให้ระยะเวลาเพิ่มขึ้นโดยไม่มีขอบเขต และเมื่อ REGISTER สำเร็จอีกครั้ง ตัวจับเวลาจะถูกรีเซ็ตกลับไปเริ่มต้นที่ 1 วินาที

แนวทางนี้เหมาะสมกับสถานการณ์ของ Sensor Node ในพื้นที่จริงที่อาจเกิดปัญหาเครือข่ายชั่วคราว และช่วยป้องกันการเกิด Reconnect Storm ในกรณีที่เซิร์ฟเวอร์หยุดให้บริการเป็นระยะเวลาหนึ่ง

7.3 การยกเลิกการเชื่อมต่อโดยตั้งใจ

การได้รับ SIGINT หรือคำสั่ง SHUTDOWN จะถือว่าเป็นการยกเลิกการเชื่อมต่อโดยตั้งใจ (Intentional Disconnect) Client จะกำหนดสถานะการยกเลิกการเชื่อมต่อเป็น Intentional และส่ง UNREGISTER ก่อนปิด Connection ทำให้กลไก Auto-Reconnect ไม่ถูกเรียกใช้งานในกรณีที่ผู้ใช้หรือเซิร์ฟเวอร์เป็นผู้สั่งให้หยุดการทำงานโดยเจตนา

หมายเหตุด้านการออกแบบ

เนื่องจากทั้งสองฝั่งจะรีเซ็ตตัวนับ Seq พร้อมกันทุกครั้งที่เกิด REGISTER ใหม่ จึงไม่ควรเกิด Gap ของลำดับข้อมูลระหว่างการทำงานปกติ ดังนั้น หน้าที่หลักของตัวตรวจจับ Seq คือการตรวจจับ Client ที่ส่งข้อมูลผิดรูปแบบหรือมีพฤติกรรมที่ผิดปกติ เช่น Client ที่มีข้อผิดพลาดหรือพยายามส่งข้อมูลที่ไม่ถูกต้อง ซึ่งมีลักษณะเดียวกับการตรวจสอบ Version นี่เป็น คุณสมบัติที่เกิดขึ้นโดยตั้งใจจากการออกแบบ (Design Property) ไม่ใช่ข้อผิดพลาดของระบบ เนื่องจากตัวตรวจจับความผิดปกติจะถูกใช้งานจริงเมื่อได้รับ Input ที่ผิดปกติหรือถูกสร้างขึ้นโดยเจตนาเท่านั้น

8. การยืนยันตัวตน (Authentication)

โดยคำเริ่มต้น VMP ไม่มีระบบ Authentication กล่าวคือ Client ใดก็ตามที่ส่ง REGISTER ในรูปแบบที่ถูกต้องตามข้อกำหนดจะได้รับการยอมรับเพื่อเพิ่มความปลอดภัยโดยไม่จำเป็นต้องจัดการ Certificate ระบบจึงรองรับการยืนยันตัวตนด้วย Shared Secret แบบ Optional โดยเซิร์ฟเวอร์สามารถเริ่มทำงานพร้อมระบุ Secret ผ่าน --secret <token> เมื่อกำหนด Secret แล้ว REGISTER จะต้องมีส่วน Header Auth-Token ที่ตรงกับ Secret ที่กำหนดไว้ หาก Token ไม่ถูกต้องหรือไม่มีการระบุมา เซิร์ฟเวอร์จะตอบกลับ 403 Forbidden ในกรณีที่ไม่ได้ระบุ --secret ระบบจะปิดการตรวจสอบ Authentication โดยสมบูรณ์ ทำให้โปรโตคอลยังคง Backward-Compatible กับการใช้งานในสภาพแวดล้อมที่ไม่มีการรักษาความปลอดภัย

มีการพิจารณาการใช้ TLS (Transport Layer Security) อย่างไรก็ตาม ได้ตัดสินใจเลื่อนการดำเนินการออกไปก่อน (§13) เนื่องจาก TLS สามารถให้การรักษาความลับของข้อมูล (Confidentiality) ได้จริง แต่ต้องมีการจัดการ Certificate และ Key เพิ่มเติม ซึ่งมีต้นทุนด้านการตั้งค่าและความซับซ้อนสูงกว่า เมื่อเทียบกับ ประโยชน์ที่ได้รับ ภายใต้ระยะเวลาที่เหลือสำหรับการพัฒนาส่วนอื่นของงานทั้งสองส่วน ดังนั้น VMP จึงเลือกใช้ Shared-Secret Authentication เป็นกลไกความปลอดภัยระดับพื้นฐาน และสงวนการนำ TLS มาใช้สำหรับการพัฒนาในอนาคต

9. การเจรจาเวอร์ชัน (Version Negotiation)

ข้อความ Start Line ของ VMP จะระบุ Version Token VMP/1.0 ไว้เสมอ แต่เดิมระบบยังไม่ได้ตรวจสอบ Version ดังกล่าว ส่งผลให้ Client ทุกตัวได้รับการประมวลผลในลักษณะเดียวกัน ไม่ว่าจะส่ง Version ใดมาก็ตาม ปัจจุบันเซิร์ฟเวอร์จะทำการ ตรวจสอบ Version Token ในทุก Request ก่อนเข้าสู่ขั้นตอน Dispatch หาก Version ที่ได้รับไม่ตรงกับ Version ที่เซิร์ฟเวอร์รองรับ ระบบจะปฏิเสธ Request และตอบกลับด้วย 400 BadRequest แทนที่จะพยายาม Parse ข้อความดังกล่าวเสมือนว่าเป็น Version ที่รองรับ การออกแบบนี้ทำให้ Version Token ซึ่งได้รับแนวคิดมาจาก HTTP ไม่ได้เป็นเพียงข้อมูลประกอบ (Decoration) แต่ทำหน้าที่เป็น Compatibility Gate ที่ใช้ตรวจสอบความเข้ากันได้ระหว่าง Client และ Server จริง

นอกจากนี้ยังช่วยรองรับการพัฒนาในอนาคต เช่น หากมีการออกแบบ VMP/2.0 ที่เปลี่ยนแปลง Wire Format ระบบสามารถปฏิเสธ Client รุ่นเก่าที่ไม่รองรับ Version ใหม่ได้อย่างชัดเจน แทนที่จะปล่อยให้ เกิดความผิดพลาดที่ไม่สามารถคาดการณ์ได้ระหว่างกระบวนการ Parse ข้อมูล

10. การระบุเป้าหมายแบบกลุ่ม (Group Addressing)

โดยทั่วไปหนึ่งแปลงเกษตรอาจมีโหนดเซนเซอร์หลายประเภทที่จำเป็นต้องได้รับคำสั่งเดียวกันพร้อมกัน เช่น ในกรณีที่คาดว่าจะเกิดพายุ ผู้ปฏิบัติงานอาจต้องการสั่งให้โหนดทุกตัวใน PLOT-01 เพิ่มความถี่ในการส่งข้อมูล ดังนั้น Operator REPL จึงรองรับคำสั่งในรูปแบบ command <target> ... โดยสามารถระบุเป้าหมายได้ 2 รูปแบบ

- Node-ID ส่ง COMMAND ไปยังโหนดที่ระบุเพียงตัวเดียว โดยยังคงพฤติกรรมเดิม
- Plot-ID ส่ง COMMAND เดียวกันไปยังโหนดทุกตัวที่ลงทะเบียนอยู่ในแปลงเกษตรนั้น

เมื่อระบุ Plot-ID เซิร์ฟเวอร์จะทำการ กระจายคำสั่ง (Fan-out) ไปยังโหนดทุกตัวที่มี Plot-ID ตรงกันและยังคงลงทะเบียนอยู่ในขณะนั้น การส่งคำสั่งจะถูกบันทึกลง Log แยกเป็นรายโหนด เพื่อให้ เป็นไปตามข้อกำหนด ของงาน ที่ต้องบันทึกทุกข้อความที่ส่งและได้รับ ซึ่งหมายความว่าแต่ละการส่ง COMMAND ไปยังแต่ละโหนดจะยังคงมี Log ของตัวเองอย่างครบถ้วน

แนวทางนี้ให้ผลลัพธ์ในทางปฏิบัติใกล้เคียงกับการใช้ Topic-based Group Addressing ของ MQTT แต่ไม่จำเป็นต้องเพิ่ม Message Broker เข้ามาในระบบ เนื่องจากเซิร์ฟเวอร์ VMP มีข้อมูล Plot-ID ของแต่ละโหนดอยู่แล้ว ภายใน Connection Table ดังนั้น Group Addressing จึงสามารถนำกลไกที่มีอยู่แล้วมาใช้เพื่อรองรับการส่งคำสั่งแบบกลุ่ม โดยไม่เพิ่มองค์ประกอบหรือความซับซ้อนของระบบโดยไม่จำเป็น

11. หลักเกณฑ์การตั้งชื่อ (Naming Conventions)

VMP กำหนดรูปแบบการตั้งชื่อสำหรับ Plot-ID, Node-ID และ Timestamp เพื่อให้ข้อมูล มีรูปแบบที่สม่ำเสมอ และสามารถตรวจสอบความถูกต้องได้ ดังนี้

ชื่อ (Name)	รูปแบบ (Format)	ตัวอย่าง (Example)
Plot-ID	PLOT-{2 digits}	PLOT-01
Node-ID	{TYPE-PREFIX}-{2 digits}	TEMP-01 / SOIL-01 / LIGHT-01
Timestamp (PUSH Body)	ISO 8601 พร้อม Bangkok Timezone Offset	2026-08-30T14:30:00+07:00

สำหรับ Node-ID จะต้องสอดคล้องกับ Node-Type ที่โหนดประกาศไว้ โดย Prefix ของ Node-ID ต้องตรงกับประเภทของโหนด ดังนี้

Node-Type	Node-ID Prefix
TempHumidNode	TEMP-
SoilNode	SOIL-
LightNode	LIGHT-

หาก Node-Type และ Prefix ของ Node-ID ไม่ตรงกัน เซิร์ฟเวอร์จะปฏิเสธ REGISTER และตอบกลับด้วย 400 BadRequest

การตรวจสอบดังกล่าวช่วยป้องกันการระบุประเภทของโหนดที่ไม่สอดคล้องกับ Identity ที่ประกาศไว้ และทำให้ข้อมูลภายในระบบมีรูปแบบที่ชัดเจนและตรวจสอบได้ง่ายขึ้น

12. การเปรียบเทียบกับ MQTT / CoAP

VMP ไม่ได้มีวัตถุประสงค์เพื่อทดแทน MQTT หรือ CoAP สำหรับการใช้งานจริง (Production) แต่เป็นโพรโทคอลที่ออกแบบมาโดยเฉพาะเพื่อให้สอดคล้องกับข้อจำกัดและข้อกำหนดของงานนี้ ได้แก่ การสื่อสารแบบ Hybrid Push + Command ผ่าน TCP Connection เดียว การบันทึกข้อความทุกข้อความในรูปแบบที่มนุษย์สามารถอ่านได้ และการรองรับโหนด 3 ประเภทที่มีชุดข้อมูลแตกต่างกัน

MQTT และ CoAP ยังคงเป็นโพรโทคอลที่เหมาะสมสำหรับใช้เป็นจุดอ้างอิงในการเปรียบเทียบ เนื่องจากทั้งสองถูกออกแบบมาเพื่อแก้ปัญหาในลักษณะเดียวกัน คือ การส่งข้อมูล Sensor Telemetry ไปยังจุดศูนย์กลาง พร้อมรองรับการส่งคำสั่งควบคุมกลับไปยังอุปกรณ์

เปรียบเทียบ	VMP	MQTT	CoAP
Transport	TCP; ตรวจสอบ Gap/Duplicate ด้วย Seq; Auto-reconnect พร้อม Backoff	TCP + รองรับ QoS 0/1/2	UDP; ความน่าเชื่อถือจัดการโดย CoAP ผ่าน Confirmable Messages
Encoding	Text, Header รูปแบบคล้าย HTTP + JSON Body	Binary, Fixed Header ขั้นต่ำ 2 ไบต์	Binary, Fixed Header 4 ไบต์ มักใช้ Payload แบบ CBOR/SenML
Framing	Content-Length ในรูปแบบ HTTP	Binary Variable-length Remaining-Length	1 UDP Datagram = 1 Message
รูปแบบการสื่อสาร	Hybrid Push + Command, Connection เดียว, ไม่มี Broker; รองรับ Group Addressing ด้วย Plot-ID	Pub/Sub ผ่าน Central Broker และ Topic Hierarchy	REST-style Request/Response; Server Push รองรับผ่านส่วนขยาย Observe
Identity / Session	Node-ID ถูกผูกกับ Socket ฟังก์ชันเวอร์ชัน; ตรวจสอบ Version; รองรับ Shared-secret Authentication แบบ Optional	Client-ID + Session State ที่ Broker จัดการ	Stateless; Request/Response จับคู่ด้วย Token ประจำ Exchange
ความยืดหยุ่นของ Schema	JSON Body สามารถมีโครงสร้างแตกต่างกันตามประเภทของโหนด	Payload เป็นข้อมูลทึบ (Opaque) โดย Schema เป็นข้อตกลงในระดับ Application	Payload เป็นข้อมูลทึบ (Opaque) เช่นเดียวกับ MQTT

เปรียบเทียบ	VMP	MQTT	CoAP
ความสามารถในการอ่านโดยมนุษย์	อ่านได้โดยตรงจาก Terminal Log โดยไม่ต้องใช้ Decoder	Binary ต้องใช้ Dissector/CLI Tool	Binary ต้องใช้ Dissector/CLI Tool
Footprint	หนักที่สุดใน 3 ตัว มี JSON และ Text Header	ขนาดเล็ก ออกแบบมาสำหรับอุปกรณ์ที่มีข้อจำกัดและใช้แบตเตอรี่	เล็กที่สุด ออกแบบมาสำหรับอุปกรณ์ระดับ 6LoWPAN

การแลกเปลี่ยนระหว่างประสิทธิภาพและความสามารถในการตรวจสอบ

VMP ยอมแลกประสิทธิภาพด้านขนาดและ Overhead อย่างมีเจตนา เนื่องจากข้อกำหนดของงานระบุว่าต้องแสดงทุก Message และ Status Code ในรูปแบบที่มนุษย์สามารถอ่านได้ หากใช้ Binary Protocol จะต้องพัฒนา Decoder เพิ่มเติมเพียงเพื่อรองรับข้อกำหนดด้าน Logging ซึ่งไม่ได้ให้ประโยชน์เพิ่มเติมที่มันยสำคัญสำหรับระบบที่มีขนาดเล็กในระดับนี้

นอกจากนี้ ระบบยังไม่มีมีความจำเป็นต้องใช้ MQTT Broker เนื่องจาก มีเพียงโหนดจำลองจำนวนไม่มาก ที่ทำงานอยู่บนเครื่องเดียวกัน ไม่ใช่ระบบขนาดใหญ่ที่ต้องการการแยกส่วนระหว่าง Publisher และ Subscriber

ในทำนองเดียวกัน ระบบไม่มีความจำเป็นต้องลด Footprint ไปถึงระดับของ CoAP/6LoWPAN เนื่องจากโหนดในระบบนี้เป็น Process ทั่วไป ไม่ใช่ Microcontroller ที่มี ข้อจำกัดด้านหน่วยความจำหรือพลังงานอย่างเข้มงวด

จุดแข็งที่ออกแบบมาเฉพาะสำหรับ VMP

- ตรวจจับข้อมูลสูญหายและข้อมูลซ้ำหลังการ Reconnect ใ้ร่วมกับระบบ Auto-reconnect โดย MQTT ใช้ QoS ที่จัดการผ่าน Broker ขณะที่ CoAP ใช้ Confirmable Messages ส่วน VMP สามารถ ตรวจจับความผิดปกติดังกล่าวได้โดยไม่ต้องใช้ Broker หรือเปลี่ยน Transport Protocol
- รองรับ Optional Authentication ในตัว สามารถเปิดใช้งาน Shared-secret Authentication ได้เมื่อจำเป็น และไม่มี Overhead เมื่อไม่ได้ใช้งาน
- Version Token ที่มีการบังคับใช้จริง VMP/1.0 ไม่ได้เป็นเพียงข้อมูลประกอบ แต่ถูกตรวจสอบเพื่อใช้เป็น Compatibility Gate
- Group Addressing โดยไม่ต้องใช้ Broker CoAP ไม่มีความสามารถในลักษณะนี้โดยตรง ขณะที่ MQTT สามารถทำได้ผ่าน Topic และ Broker ส่วน VMP ใช้ Plot-ID และ Connection Table ที่มีอยู่แล้ว
- ตรวจสอบและ Debug ได้โดยไม่ต้องใช้ Decoder ทุกข้อความสามารถอ่านได้โดยตรงจาก Log ซึ่งเป็นสิ่งที่ระบบจำเป็นต้องบันทึกอยู่แล้วตามข้อกำหนดของงาน

13. งานที่เลื่อนไว้สำหรับการพัฒนาในอนาคต (Deferred Future Work)

มีการพิจารณาความสามารถเพิ่มเติมหลายรายการ แต่ตัดสินใจไม่นำมาใช้ใน Submission ฉบับนี้ เนื่องจากต้นทุนและความซับซ้อนในการพัฒนาสูงเกินความเหมาะสมเมื่อเทียบกับประโยชน์ที่ได้รับ ภายใต้ระยะเวลาที่มีอยู่ได้แก่

- Full Session Resumption ให้เซิร์ฟเวอร์ออก Session Token เพื่อให้โหนดสามารถกลับมาใช้สถานะเดิมได้หลังจาก Reconnect โดยไม่จำเป็นต้องทำ REGISTER ใหม่ตั้งแต่ต้น
- Message-ID + Dedupe กำหนด Unique ID ให้กับแต่ละ Message และให้เซิร์ฟเวอร์ตรวจสอบและป้องกันการประมวลผล Message ซ้ำ ซึ่งมีความซับซ้อนกับความสามารถในการตรวจจับ Duplicate ที่ Seq รองรับอยู่แล้วในระดับหนึ่ง
- Batch PUSH อนุญาตให้ส่งข้อมูลที่ตรวจวัดได้หลายรายการรวมอยู่ใน Message เดียว เพื่อลด Overhead ในกรณีที่มีการส่งข้อมูลในช่วงเวลาสั้น ๆ อย่างไรก็ตาม ไม่จำเป็นสำหรับจำนวนโหนดที่ใช้ในโครงการนี้
- TLS เพิ่มการเข้ารหัสการสื่อสารในระดับ Transport เพื่อทดแทนการใช้ Shared-secret Header แต่ตัดสินใจเลื่อนออกไป เนื่องจากการจัดการ Certificate และ Key เพิ่มความเสี่ยง ด้านการตั้งค่า และใช้เวลาในช่วงการตรวจงานมากเกินไปเมื่อเทียบกับประโยชน์ที่ได้รับในขอบเขตของโครงการนี้

โดยรวมแล้ว การตัดความสามารถเหล่านี้ ออกไม่ได้หมายความว่า เป็นข้อจำกัดถาวรของ VMP แต่เป็นการกำหนดขอบเขตของงาน (Scope Management) เพื่อให้สามารถ มุ่งเน้นไปที่ฟังก์ชันหลัก ของโพรโทคอลและส่งมอบระบบ ที่ทำงานได้ครบถ้วนภายในเวลาที่กำหนด